

Хук useCallback



Статус

Готово

Хук useCallback

У цьому розділі розглянемо рідко використовувані або нішеві React-хуки. Вони не є частиною обов'язкової бази, а скоріше додатковим матеріалом для розширення кругозору. Достатньо знати про їхнє існування та базове призначення. Глибоко занурюватися не обов'язково — повернетеся до них за потреби.

У React щоразу, коли компонент рендериться повторно, **всі функції, оголошені всередині компонента, створюються заново**. Це зазвичай не є проблемою, але іноді створення нової функції може викликати небажані побічні ефекти або зайві рендери дочірніх компонентів.

Хук `useCallback` допомагає уникнути таких ситуацій. Він **memoізує** функцію — тобто повертає одну й ту саму версію функції між рендерами, **якщо не змінилися залежності**.

```
import { useCallback } from 'react';
```

Що робить useCallback?

Хук `useCallback(fn, deps)` повертає **memoізовану версію функції** `fn`, яка буде змінюватись тільки тоді, коли зміниться хоча б одна з залежностей у масиві `deps`.

```
const memoizedCallback = useCallback(() => {  
  doSomething(a, b);  
}, [a, b]);
```

`useMemo` використовується для мемоізації результату обчислення, а `useCallback` — для мемоізації самої функції.

Якщо простіше: `useMemo` = «запам'ятай значення», `useCallback` = «запам'ятай функцію».

Навіщо це потрібно?

Розглянемо ситуацію: у нас є компонент `Parent`, у якому є стан і функція, яка передається до дочірнього компонента `Child`.

```
// src/components/Parent.tsx  
  
import { useState } from 'react';  
import Child from './Child.tsx';  
  
export default function Parent() {  
  const [count, setCount] = useState(0);  
  
  const handleClick = () => {  
    console.log('Clicked!');  
  };  
  
  return (  
    <>  
      <button onClick={() => setCount(count + 1)}>Count: {count}</button>  
      <Child onClick={handleClick} />  
    </>  
  );  
}
```

```
// src/components/Child.tsx

export default function Child({ onClick }: { onClick: () => void }) {
  console.log('Child render');

  return <button onClick={onClick}>Click me</button>;
}
```

- У цьому прикладі кнопка «Count» збільшує лічильник.
- Компонент `Child` не залежить від `count`, але **все одно буде рендеритись при кожному оновленні `Parent`**, тому що `handleClick` — нова функція на кожному рендері.

Це не критично, але якщо `Child` — важкий компонент, або в нього буде `React.memo`, ми не зможемо уникнути його повторного рендеру без `useCallback`.

Оптимізуємо з `useCallback`

```
// src/components/Parent.tsx

import { useState, useCallback } from 'react';
import Child from './Child.tsx';

export default function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    console.log('Clicked!');
  }, []);

  return (
    <>
      <button onClick={() => setCount(count + 1)}>Count: {count}</button>
      <Child onClick={handleClick} />
    </>
  );
}
```

```
    </>
  );
}
```

Тепер `handleClick` не буде створюватися заново на кожному рендері. Це відкриває можливість **уникнути повторного рендеру дочірнього компонента**, якщо обгорнути його в `React.memo`.

`React.memo` — це спеціальна функція, яка дозволяє React "запам'ятати" результат рендеру компонента. Якщо пропси не змінились, компонент не буде рендеритись повторно.

```
// src/components/Child.tsx

import { memo } from 'react';

interface Props {
  onClick: () => void;
}

function Child({ onClick }: Props) {
  console.log('Child render');

  return <button onClick={onClick}>Click me</button>;
}

export default memo(Child);
```

У нашому випадку, якщо `onClick` не змінюється (memoізована за допомогою `useCallback`), то `Child` не буде рендеритись повторно. Це допомагає зменшити кількість непотрібних рендерів і покращити продуктивність.

`React.memo` працює схоже на `useMemo`, але для **всього компонента**, а не просто функції чи значення. Ви побачите цей патерн часто: `React.memo` + `useCallback` → оптимізація компонентів, що отримують функції через пропси.

useCallback з залежностями

Якщо функція залежить від якихось значень (наприклад, стану), потрібно вказати їх у масиві залежностей:

```
const [count, setCount] = useState(0);

const handleClick = useCallback(() => {
  console.log(`Current count: ${count}`);
}, [count]);
```

Якщо не додати `count` до залежностей, `handleClick` "запам'ятає" старе значення і не оновиться.

Коли використовувати useCallback?

Використовуйте `useCallback`, якщо:

- Ви передаєте функцію в компонент, який обгорнутий у `React.memo`.
- Ви використовуєте функцію в `useEffect`, `useMemo` або іншому `useCallback`, і вам потрібно, щоб вона не змінювалась між рендерами.
- Створення функції викликає побічні ефекти або зайві ререндери.

Не використовуйте `useCallback` без причини. У більшості випадків створення нової функції — це дешево. Але в деяких ситуаціях — **особливо з оптимізацією великих компонентів** — мемоізація дійсно допомагає.

Підсумок

- `useCallback(fn, deps)` повертає **memoізовану версію функції** `fn`.
- Якщо залежності не змінились, повертається попередня версія функції.
- Використовується для оптимізації — **щоб уникнути створення нових функцій на кожному рендері**.
- Часто корисний у зв'язці з `React.memo` для зменшення непотрібних рендерів дочірніх компонентів.
- Не використовуйте всюди — тільки там, де це має сенс.